Pooling

Contents

1. Overview . . . 2
2. Prerequisites . . . 3
3. Why pooling is needed . . . 3
   - 3.1 Memory . . . 3
   - 3.2 Features tied to their location . . . 4
4. Max pooling . . . 5
   - 4.1 Where pooling sits . . . 5
   - 4.2 The three settings . . . 5
   - 4.3 One worked example . . . 5
5. Pooling a volume . . . 7
6. Pooling in Keras . . . 7
7. Advantages of pooling . . . 9
   - 7.1 Reduced size . . . 9
   - 7.2 Translation invariance . . . 9
   - 7.3 Enhanced features (max pooling only) . . . 11
   - 7.4 Nothing to train . . . 11
8. Types of pooling . . . 12
   - 8.1 Max, average and L2 pooling . . . 12
   - 8.2 Global pooling . . . 13
9. Disadvantages of pooling . . . 14
10. Summary . . . 15
11. Sources . . . 15
12. Key terms . . . 16

Where this fits: Pipeline map step 8 (Model). See the Course map.

0 Foundations	1 Frame	2 Get data	3 Understand	4 Clean	5 Features	6 Reduce
7 Split	**8 Model**	9 Evaluate	10 Tune	11 Deploy	12 Test	13 Monitor

- **Builds on:** Convolutional neural network (CNN); Convolution operation and feature maps.
- **Leads to:** CNN architecture (LeNet-5).

1. Overview

Key point: A pooling layer shrinks each feature map by replacing every small window with one summary number, usually its maximum. **Pooling** (G-1521) cuts memory, makes the network less sensitive to where exactly a feature sits, and has nothing to train.

A pooling layer comes right after a convolution layer (a layer that slides filters over an image, see the convolution operation) and its ReLU (the step that sets negative values to 0) in a convolutional neural network (CNN, a network for images). Pooling answers two problems of convolution: feature maps take a lot of memory, and their values are tied to the exact position of each feature.

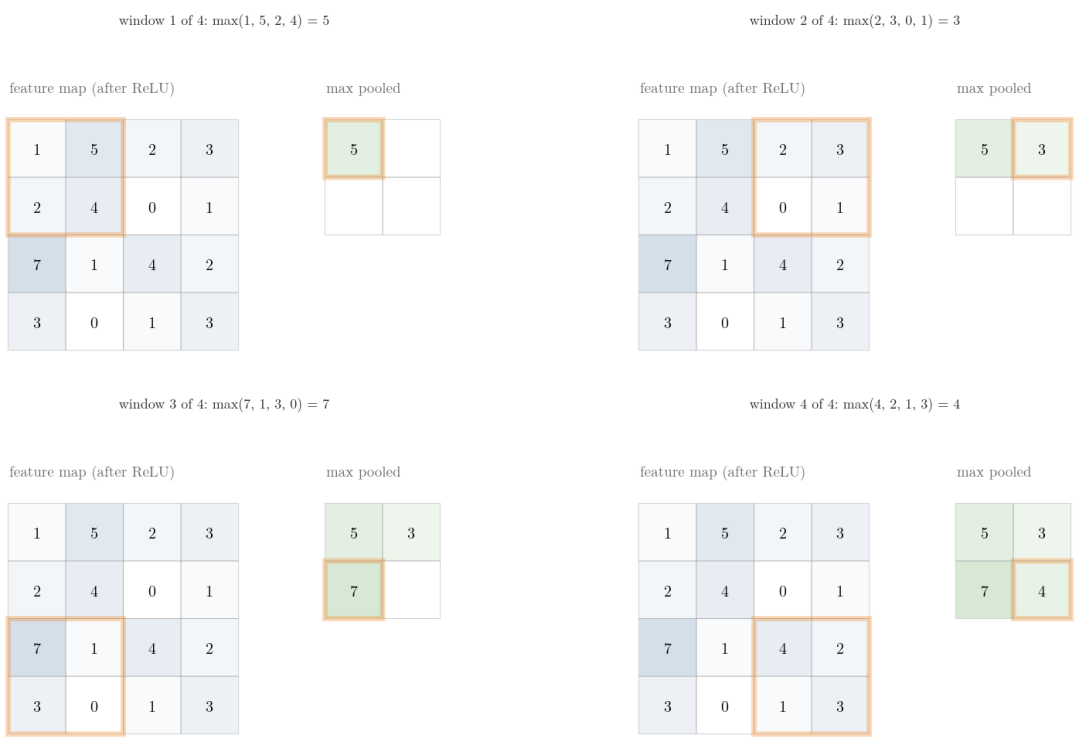


Figure 1: Max pooling with a 2 × 2 window and stride 2: each window of the 4 × 4 feature map is replaced by its largest value, giving a 2 × 2 map

Figure 1 shows the most common kind, **max pooling** (G-1182). Its window is 2×2 numbers, and its stride, the number of cells the window moves each time, is 2 (**strides** (G-1900)). This Note covers:

- the two problems (section 3);
- how max pooling works (section 4);
- pooling on volumes (section 5);
- pooling in Keras (section 6);
- its advantages (section 7);
- the other kinds of pooling (section 8);
- its disadvantages (section 9).

2. Prerequisites

- The convolution operation: feature maps, ReLU after convolution and volumes of feature maps.
- **Strides** (G-1900): the stride and the output-size formula.

3. Why pooling is needed

> **Key point:** Feature maps are large: one 224×224 colour image through 100 filters needs about 20 MB, and a batch of 100 images about 2 GB. And a convolution's output moves with the input, so the same feature at a different place gives different numbers.

3.1 Memory

> **Key point:** $222 \times 222 \times 100$ numbers of 4 bytes each: 19.7 MB per image, 1.97 GB per batch of 100.

Take an RGB image of $224 \times 224 \times 3$, a common input size for image networks (CS231n notes), and a convolution layer with 100 filters of $3 \times 3 \times 3$. Each **filter** (G-777; a small grid of weights) gives a **feature map** (G-766; the grid of numbers a filter writes) whose side is (see the size of the feature map):

$$224 - 3 + 1 = 222$$

So the layer outputs a **volume** (a stack of feature maps, one per filter) of $222 \times 222 \times 100$.

1. **In words:** count the numbers in the volume and multiply by 4 bytes, the size of one 32-bit floating-point number.
2. **Formula:**
$$\text{memory} = \text{height} \times \text{width} \times \text{filters} \times 4 \text{ bytes}$$
3. **Example:** count the numbers, then multiply by 4 bytes (Notebook):

$$222 \times 222 \times 100 = 4{,}928{,}400$$

$$4{,}928{,}400 \times 4 \text{ bytes} = 19.7 \text{ MB}$$

That is 19.7 MB for one image. Training sends a whole batch (a group of images processed together) at once: 100 images need 1.97 GB, for the output of one layer alone.

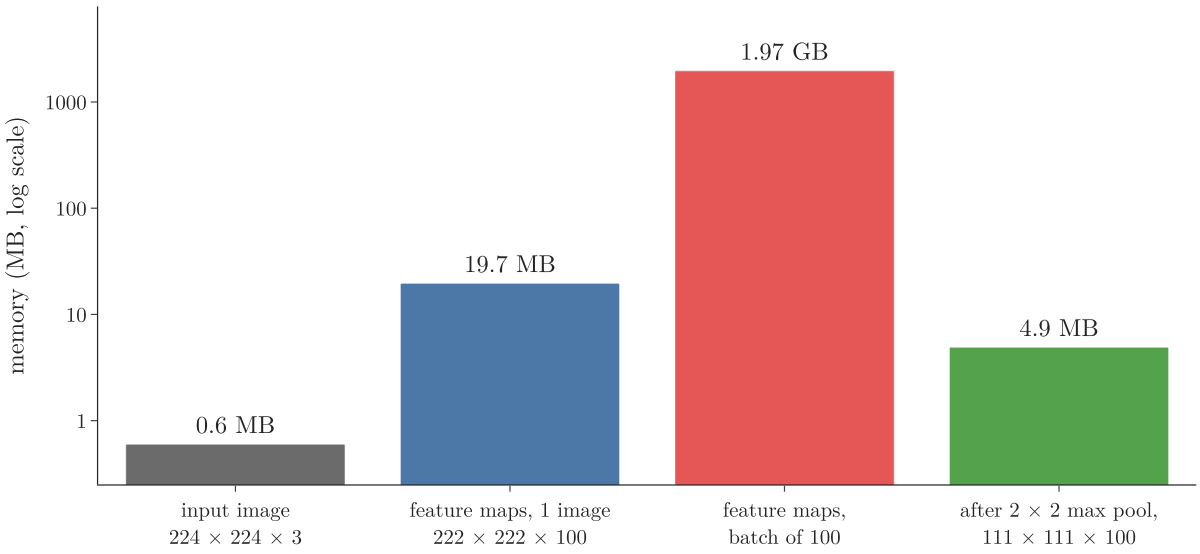


Figure 2: Memory of one input image, of the feature maps one convolution layer of 100 filters makes from it, of those maps for a batch of 100 images, and of one image's maps after 2×2 max pooling (section 7.1). Log scale; 4 bytes per number.

In Figure 2, the first two steps to the right are jumps of a power of ten or more: one convolution layer turns a 0.6 MB image into 19.7 MB of feature maps, and a batch of 100 makes that 1.97 GB. **Pooling** (G-1521) (green) brings one image's 19.7 MB back down by a factor of 4, to 4.9 MB.

Such volumes can slow a machine down or make the program crash for lack of memory. So we want to shrink the feature maps. A larger **stride** (G-1900) (see why use strides) would do it, but pooling also solves a second problem.

3.2 Features tied to their location

> **Key point:** If the input shifts, the feature map shifts with it. For classification we care whether a feature is present, not exactly where.

A convolution layer finds features such as edges, eyes or ears. Its output is tied to location: if the cat sits on the left of the image, its ear lights up the left of the feature map; if it sits on

the right, the right. In the Notebook, a digit shifted 1 pixel to the right gives exactly the same feature map, shifted 1 pixel to the right.

For a task such as telling cats from dogs, the location of the ear does not matter, only that it is an ear. But the next layers see different numbers for the same ear at different places. We would like the network to treat a feature the same way wherever it appears, a property called **translation invariance** (G-2011). **Pooling** (G-1521) gives an approximate version of it.

> **Extra:** The precise names (Goodfellow et al. 2016, §9.2 and §9.3): convolution is **equivariant** to translation (**translation equivariance**, G-2010), meaning that if the input shifts, the output shifts in the same way. Pooling makes the output approximately **invariant** to small translations, meaning that the pooled values mostly do not change. The problem of section 3.2 is therefore equivariance where invariance is wanted.

4. Max pooling

> **Key point:** Choose a window size (usually 2×2), a **stride** (G-1900) (usually 2) and a type (usually max). Slide the window over the feature map and keep the largest value in each window.

4.1 Where pooling sits

> **Key point:** Image, convolution, ReLU, then pooling.

The order inside a CNN is: convolution gives the feature map, ReLU adds non-linearity by turning the negative values into 0, and the pooling layer then shrinks the result. **Pooling** (G-1521) is a **downsampling** (G-636) operation: it reduces the size of the feature map.

4.2 The three settings

> **Key point:** Size of the window, stride, and type of **pooling** (G-1521).

A pooling layer needs three settings:

1. the **size** of the window, the area it summarises, usually 2×2;
2. the **stride** (G-1900), usually 2, so that the windows do not overlap;
3. the **type**: max, average and others (section 8).

4.3 One worked example

> **Key point:** A 4×4 map with 2×2 windows and stride 2 has 4 windows; their maxima 5, 3, 7, 4 form the 2×2 output.

1. **In words:** split the map into 2×2 blocks and keep the largest number of each block.
2. **Formula:** for output position (i, j) (row i and column j of the output, counted from 0), with window size and stride 2, $A_{r,c}$ is the number in row r and column c of the map A:

$$P_{ij} = \max\Big(A_{2i,\,2j},\ A_{2i,\,2j+1},$$

$$A_{2i+1,\,2j},\ A_{2i+1,\,2j+1}\Big)$$

3. **Example:** with the feature map of Figure 1:

$$A = \begin{bmatrix} 1 & 5 & 2 & 3 \\ 2 & 4 & 0 & 1 \\ 7 & 1 & 4 & 2 \\ 3 & 0 & 1 & 3 \end{bmatrix}$$

Take the maximum of each block:

$$P_{00} = \max(1, 5, 2, 4) = 5$$

$$P_{01} = \max(2, 3, 0, 1) = 3$$

$$P_{10} = \max(7, 1, 3, 0) = 7$$

$$P_{11} = \max(4, 2, 1, 3) = 4$$

$$P = \begin{bmatrix} 5 & 3 \\ 7 & 4 \end{bmatrix}$$

The 4×4 feature map has become 2×2. The output size follows the formula of **strides** (G-1900) without padding (Dumoulin and Visin 2016, Relationship 7):

$$\lfloor (4 - 2)/2 \rfloor + 1 = 2$$

What does the largest value mean? A feature map is large where the filter's pattern is present (see the convolution operation). So the maximum of a window marks the spot where the filter matched the image best, and **max pooling** (G-1182) keeps exactly that spot.

Within each small region, the **receptive field** (G-1642) of the output value, max pooling keeps the strongest response and drops the weaker ones. The strongest response is the most dominant feature in that region. Low-level detail is discarded, and the dominant features move on.

5. Pooling a volume

> **Key point: Pooling** (G-1521) works on each feature map separately. A $4 \times 4 \times 2$ volume becomes $2 \times 2 \times 2$; the number of channels does not change.

After a convolution with several filters we have a volume of feature maps. For example, a $6 \times 6 \times 3$ image and 2 filters of $3 \times 3 \times 3$ give $4 \times 4 \times 2$. After ReLU, pooling is applied to each feature map on its own, first the front one, then the back one, giving a $2 \times 2 \times 2$ volume. With 100 filters, a $4 \times 4 \times 100$ volume becomes $2 \times 2 \times 100$.

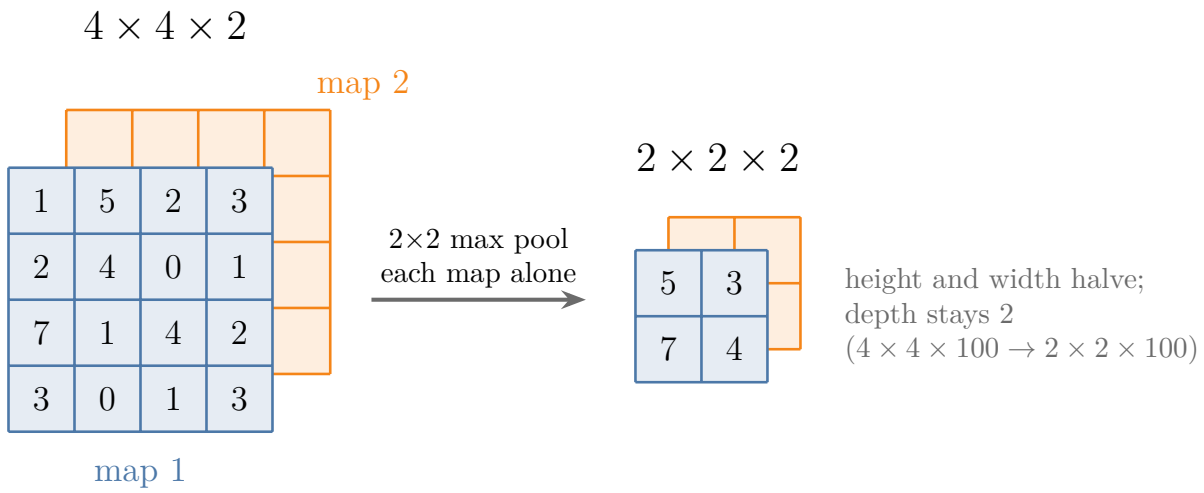


Figure 3: Pooling a $4 \times 4 \times 2$ volume: each feature map is pooled on its own. The front map is the example of section 4.3.

Pooling changes the height and width, never the depth (Figure 3). The Notebook checks that Keras' pooled volume equals pooling each channel by hand.

6. Pooling in Keras

> **Key point:** `MaxPooling2D(pool_size=(2, 2), strides=2)` (G-116) after each convolution layer. **Pooling** (G-1521) layers have 0 parameters.

Python: Two convolution layers, each followed by **max pooling** (G-1182).

```
model = keras.Sequential([
    keras.Input(shape=(28, 28, 1)),
    keras.layers.Conv2D(32, 3, activation="relu"),
    keras.layers.MaxPooling2D(pool_size=(2, 2), strides=2),
    keras.layers.Conv2D(32, 3, activation="relu"),
    keras.layers.MaxPooling2D(pool_size=(2, 2), strides=2),
    keras.layers.Flatten(),
```

```
    keras.layers.Dense(10, activation="softmax")])
```

The shapes and parameter counts from `model.summary()` (Notebook):

Layer	Output shape	Parameters
Conv2D, 32 filters 3×3	$26 \times 26 \times 32$	320
MaxPooling2D 2×2	$13 \times 13 \times 32$	0
Conv2D, 32 filters 3×3	$11 \times 11 \times 32$	9,248
MaxPooling2D 2×2	$5 \times 5 \times 32$	0
Flatten	800	0
Dense, 10 nodes	10	8,010

Each max pooling layer halves the height and width. A side of 11 becomes 5:

$$\lfloor (11 - 2)/2 \rfloor + 1 = 5$$

Every pooling layer has 0 trainable parameters: taking a maximum is a fixed aggregate operation with nothing to learn. In Keras, `strides` defaults to the pool size, so `MaxPooling2D(2)` is the same layer (Keras documentation).

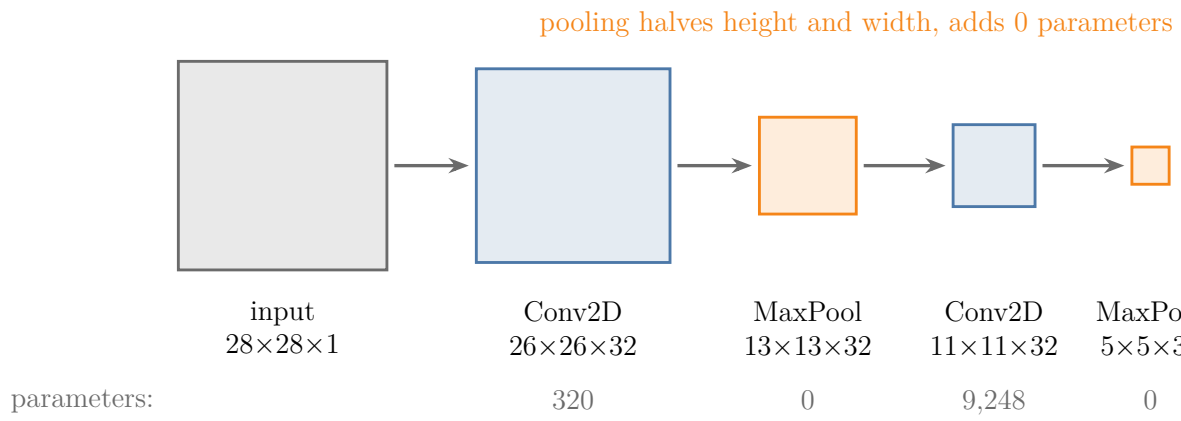


Figure 4: The output of each layer of the model above, drawn to scale (side = height of the map), with its parameter count.

In Figure 4, watch the orange squares: each pooling layer halves the side of the map and adds 0 parameters, while each convolution layer trims only 2 pixels and carries trainable weights.

7. Advantages of pooling

> **Key point:** Smaller feature maps, approximate translation invariance, stronger dominant features (**max pooling** (G-1182) only), and no parameters to train.

7.1 Reduced size

> **Key point:** 2×2 **pooling** (G-1521) with stride 2 keeps a quarter of the values: $222 \times 222 \times 100$ becomes $111 \times 111 \times 100$, 4.9 MB instead of 19.7 MB.

In the example of section 3.1, 2×2 **max pooling** (G-1182) turns the $222 \times 222 \times 100$ volume into $111 \times 111 \times 100$, a quarter of the values and of the memory (Notebook). The next layers also have less work to do.

7.2 Translation invariance

> **Key point:** On 1,000 digits shifted by 1 pixel, the feature map changes by 0.88 (relative), the 2×2 max-pooled map by 0.65, the 4×4 by 0.42, and the global maximum not at all.

Downsampling (G-636) keeps the dominant feature of each region and forgets exactly where in the region it was. So a small shift of the input changes the pooled map much less than the feature map.

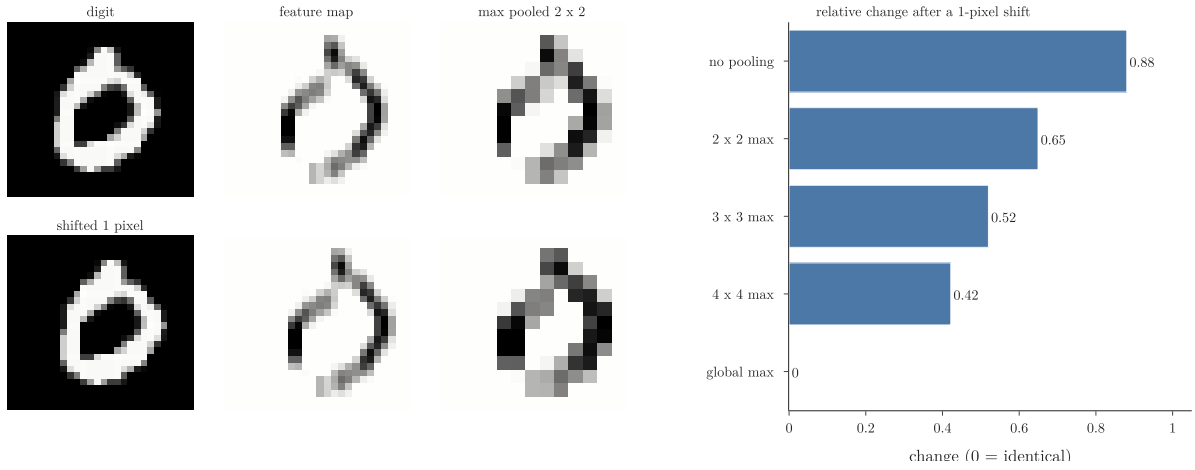


Figure 5: Left: a digit and the same digit shifted 1 pixel right, their feature maps (vertical-edge filter and ReLU) and 2×2 max-pooled maps. Right: the relative change caused by the shift, averaged over 1,000 test digits; 0 means identical

Figure 5 and the Notebook measure it on 1,000 MNIST test digits, each shifted 1 pixel to the right. The relative change is the size of the difference between the representations of the original and the shifted digit, divided by the size of the original's representation:

Representation	Relative change after a 1-pixel shift
Feature map, no pooling	0.88
After 2 × 2 max pooling	0.65
After 3 × 3 max pooling	0.52
After 4 × 4 max pooling	0.42
After global max pooling	0.00

Larger **pooling** (G-1521) windows give more invariance, and a single maximum over the whole map (section 8.2) does not change at all. Pooling makes the network focus on whether a feature is present and less on where it is.

The invariance holds for small shifts only. Figure 6 moves one digit 8 further and further to the right. Watch the feature map slide with the digit, while the pooled map keeps roughly the same dark cells at a shift of 1 pixel and then starts to change too.

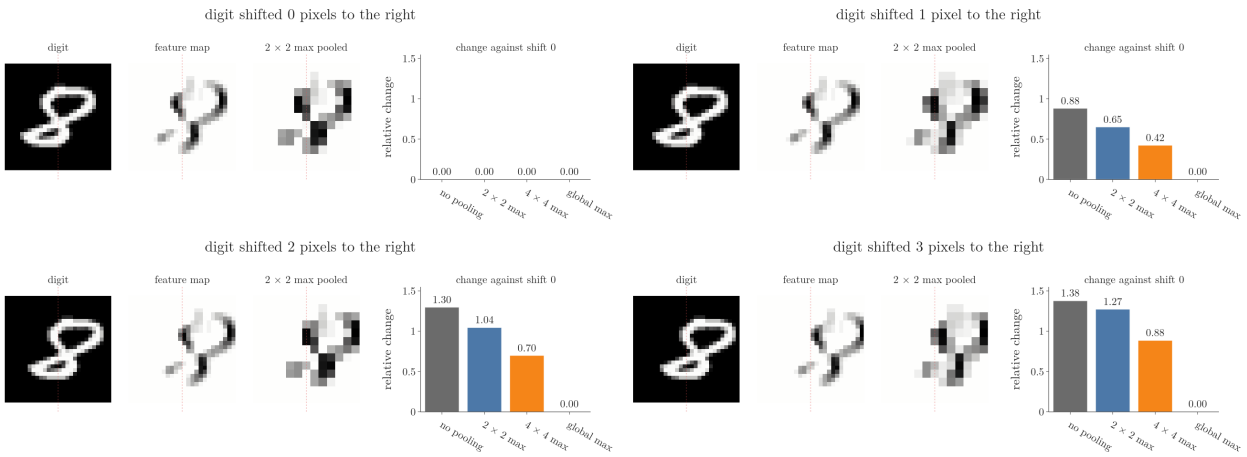


Figure 6: One MNIST 8 shifted 0, 1, 2 and 3 pixels to the right (the dotted red line stays fixed): the digit, its feature map and its 2 × 2 max-pooled map. Bars: the relative change of each representation against the unshifted digit, averaged over 1,000 test digits

Shift	No pooling	2 × 2 max	4 × 4 max	Global max
1 pixel	0.88	0.65	0.42	0.00
2 pixels	1.30	1.04	0.70	0.00
3 pixels	1.38	1.27	0.88	0.00

At every shift the pooled maps change less than the feature map, and a larger window changes less than a smaller one. But a 2 × 2 window cannot hide a shift of 2 or 3 pixels: its change climbs from 0.65 to 1.04 and 1.27 (Notebook). Pooling absorbs small shifts, not large ones.

7.3 Enhanced features (max pooling only)

> **Key point: Max pooling** (G-1182) keeps the strongest value of each window, so the edges in a pooled edge map look bolder; average pooling dilutes them.

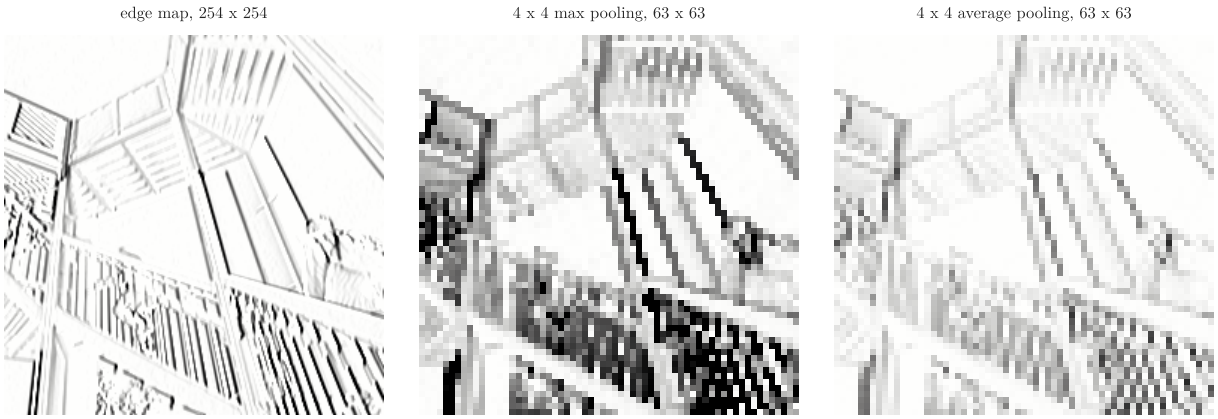


Figure 7: The vertical-edge map of a photo (left), after 4 × 4 max pooling (middle) and after 4 × 4 average pooling (right), on the same colour scale. Max pooling keeps the edges strong; averaging fades them

Each window of an edge map holds a few strong edge values and many weak ones. Max pooling keeps the strong one, so the pooled map shows the edges stronger and bolder (Figure 7, middle, where darker means a stronger edge). **Average pooling** (G-238) mixes the strong values with the weak ones, and the edges fade (right). In the Notebook the mean value of the edge map is 24.7; after max pooling it is 77.8, after average pooling 24.6. This enhancement only happens with max pooling.

7.4 Nothing to train

> **Key point:** A pooling layer only needs its settings. A pooling layer has no weights, so backpropagation has nothing to update in it.

The values of a convolution filter are learned by backpropagation. **Pooling** (G-1521) needs no training: we only choose the window size, the stride and the type, and finding the largest of 4 numbers needs no learning. With nothing to update, a pooling layer adds no parameters and no training cost.

8. Types of pooling

> **Key point: Max pooling** (G-1182) is the most used; average pooling takes the mean; global pooling reduces each whole feature map to one number.

8.1 Max, average and L2 pooling

> **Key point:** On the windows of section 4.3: max gives 5, 3, 7, 4; average gives 3, 1.5, 2.75, 2.5.

Max pooling (G-1182) keeps the largest value of each window. **Average pooling** (G-238) keeps the mean of the window instead: add the four values and divide by 4. In Figure 8, watch the same window fill both maps: the first window, 1, 5, 2, 4, gives 5 in the max-pooled map. In the average-pooled map it gives:

$$(1 + 5 + 2 + 4)/4 = 12/4 = 3$$

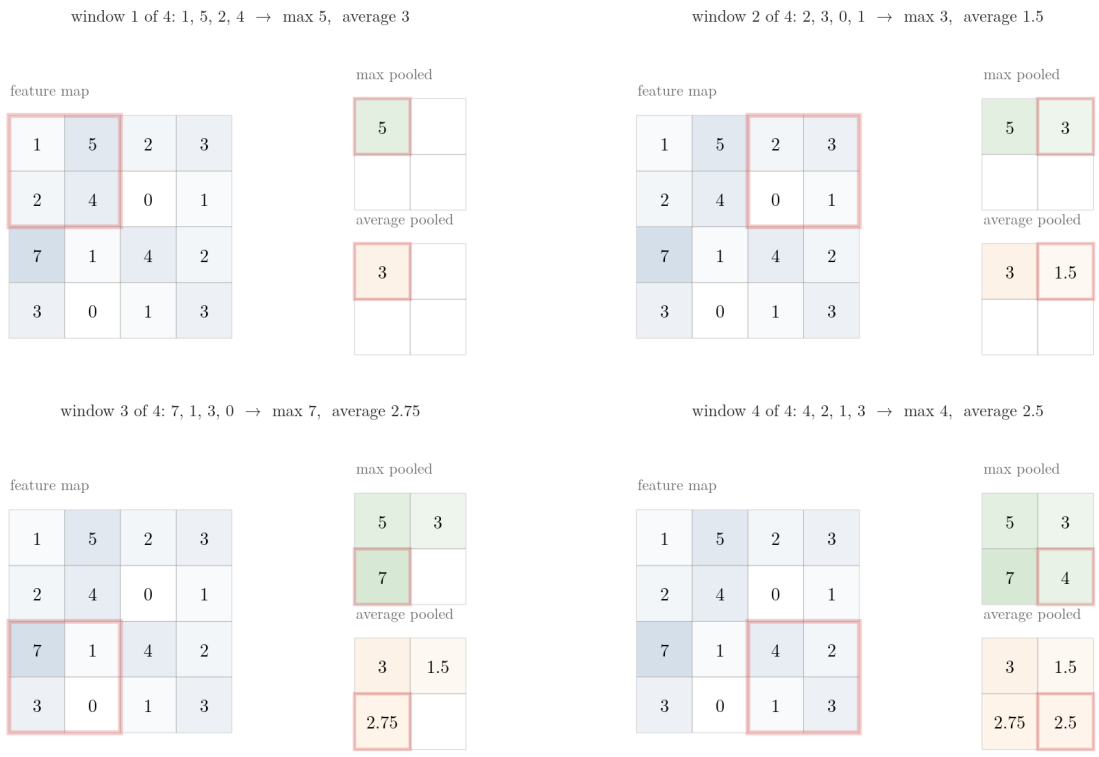


Figure 8: The map of section 4.3 pooled two ways at once. The same 2 × 2 window (red) fills the max-pooled map (green) and the average-pooled map (orange)

The window can be summarised in other ways too (Goodfellow et al. 2016, §9.3):

Type	Summary of each window	On the example of section 4.3
Max pooling	largest value	5, 3, 7, 4
Average pooling	mean value	3, 1.5, 2.75, 2.5
L2 pooling	$\sqrt{\text{sum of squares}}$	6.78, 3.74, 7.68, 5.48

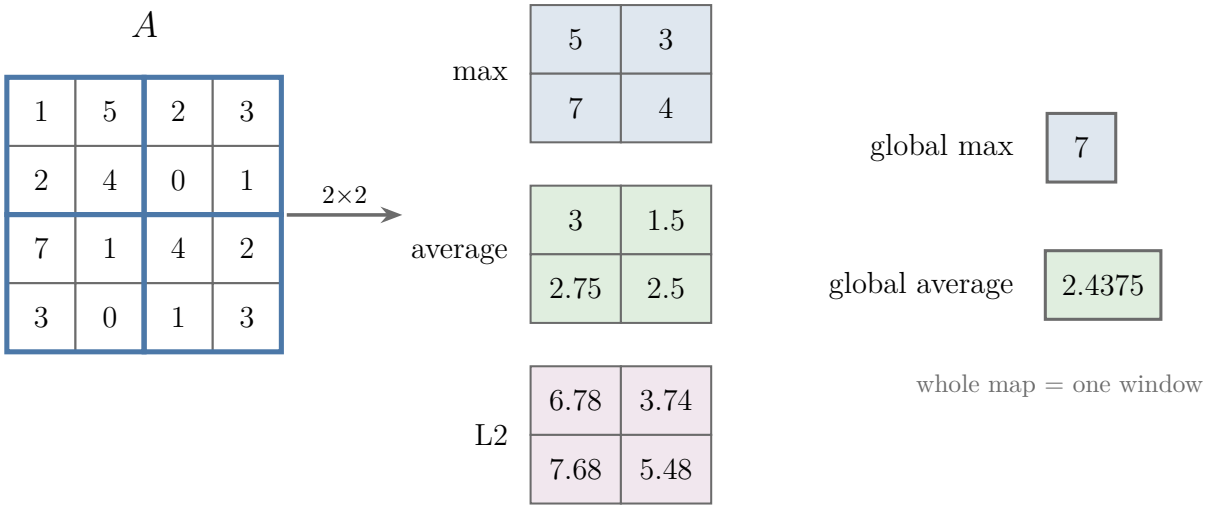


Figure 9: The map of section 4.3 summarised by max, average and L2 pooling (2 × 2, stride 2), and by global max and global average pooling.

Figure 9 puts the summaries side by side on the same map: max keeps the peaks 5, 3, 7, 4, average dilutes them, and the global versions squeeze the whole map into a single number.

Min **pooling** (G-1521), the smallest value, is also possible but is not a Keras layer. Keras has `MaxPooling2D` and `AveragePooling2D`, and the Notebook checks that both give the numbers above. Max pooling is used most; average pooling can be worth trying in some problems.

8.2 Global pooling

> **Key point:** Global pooling uses the whole feature map as one window: one number per map. A 4 × 4 × 3 volume gives 3 numbers.

Global pooling (G-849) treats the whole feature map as one window. **Global max pooling** keeps the largest value of the whole feature map; for the map of section 4.3 that is 7. **Global average pooling** takes the mean of the whole map: 2.4375. With several feature maps, each gives one number: a 4 × 4 × 3 volume becomes a vector (a list) of 3 values.

Global pooling can replace the Flatten layer before the fully connected layers. For a 5 × 5 × 32 volume, Flatten gives 800 values and a `Dense(10)` layer on them has 8,010 parameters; global **average pooling** (G-238) gives 32 values and the same layer has 330 (Notebook). Fewer parameters reduce overfitting.

Extra: Global average pooling as a replacement for the fully connected part was introduced by Lin et al. (2014), who describe it as "itself a structural regularizer, which natively prevents overfitting", whereas fully connected layers "are prone to overfitting".

9. Disadvantages of pooling

Key point: Pooling (G-1521) throws away location, which hurts tasks where location matters, and it discards 75% of the values.

1. **Lost location. Translation invariance** (G-2011) helps when only the presence of a feature matters, as in image classification. In some computer vision tasks the location is the answer, such as **image segmentation** (G-2268), which divides an image into regions and must say exactly which pixels belong to the car. There, pooling can hurt: if a task relies on preserving precise spatial information, pooling on all features can increase the training error (Goodfellow et al. 2016, §9.4).
2. **Lost information.** 2×2 pooling with **stride** (G-1900) 2 turns 16 values into 4: it keeps a quarter and discards exactly 75% of the values (CS231n notes).

Whether to pool depends on the application.

Extra: Some work drops pooling altogether. Springenberg et al. (2015) found that "max-pooling can simply be replaced by a convolutional layer with increased stride without loss in accuracy" on several image recognition benchmarks, and built a network of convolution layers only.

10. Summary

	Max pooling	Average pooling	Global pooling
Summary of a window	largest value	mean	one value per whole map
Typical setting	2×2, stride 2	2×2, stride 2	whole map
Keras layer	`MaxPooling2D`	`AveragePooling2D`	`GlobalMaxPooling2D`, `GlobalAveragePooling2D`
Parameters	0	0	0

- **Pooling** (G-1521) follows convolution and ReLU, and downsamples each feature map separately, so it changes the height and width, never the depth.
- It solves two problems: large feature maps (memory) and features tied to location, because a batch of feature maps can need gigabytes and classification cares whether a feature is present, not exactly where.
- Advantages: smaller maps, approximate **translation invariance** (G-2011), stronger dominant features (max only), no training, because each window keeps only one summary of its values and forgets where in the window the feature was.
- Disadvantages: location is lost (bad for segmentation-like tasks) and 75% of the values are discarded, so whether to pool depends on the task.
- **Global pooling** (G-849) turns each feature map into one number and can replace Flatten, so the dense layer after it needs far fewer parameters, which reduces overfitting.

11. Sources

Built from

- CampusX, "Pooling Layer in CNN | MaxPooling in Convolutional Neural Network", YouTube, https://www.youtube.com/watch?v=DwmGefkowCU
- StatQuest with Josh Starmer, "Neural Networks Part 8: Image Classification with Convolutional Neural Networks (CNNs)", YouTube, https://www.youtube.com/watch?v=HGwBXDKFk9I (max pooling keeps the spot where the filter matched best; average pooling)

Other references

- Dumoulin, V. and Visin, F. (2016). A guide to convolution arithmetic for deep learning. arXiv:1603.07285. Relationship 7 (pooling arithmetic).
- Goodfellow, I., Bengio, Y. and Courville, A. (2016). *Deep Learning*. MIT Press. §9.2 (equivariance), §9.3 (pooling, invariance to small translations, types of pooling), §9.4 (pooling can cause underfitting).
- Keras API documentation: `MaxPooling2D`, `AveragePooling2D`, `GlobalMaxPooling2D` layers, keras.io/api/layers/pooling_layers.
- Lin, M., Chen, Q. and Yan, S. (2014). Network In Network. ICLR 2014. arXiv:1312.4400.
- Springenberg, J. T., Dosovitskiy, A., Brox, T. and Riedmiller, M. (2015). Striving for Simplicity: The All Convolutional Net. ICLR 2015 workshop. arXiv:1412.6806.

- Stanford CS231n course notes, "Convolutional Neural Networks", cs231n.github.io/convolutional-networks.

12. Key terms

Terms taught in this Note come first; linked terms are recaps, taught in the Note the link opens.

Term	Meaning
Pooling (G-1521)	Replacing each small window of a feature map with one summary number, to shrink the map.
Max pooling (G-1182)	Pooling that keeps only the largest value in each window (usually 2×2), which shrinks the feature map while keeping the strongest response in each region.
Translation invariance (G-2011)	The output stays (almost) the same when the input is shifted slightly.
Translation equivariance (G-2010)	The output shifts in the same way as the input; a property of convolution.
Downsampling (G-636)	Reducing the height and width of a feature map.
`MaxPooling2D` (G-116)	The Keras layer for 2D max pooling, `MaxPooling2D(pool_size, strides)`: it keeps only the largest value in each window of a feature map, which shrinks the map; it has no parameters.
Average pooling (G-238)	Pooling that replaces each window of a feature map with the mean of its values, which shrinks the map; unlike max pooling, it fades strong values such as edges.
Global pooling (G-849)	Pooling that uses the whole feature map as one window, giving one number per map (its max or its average). It can replace the Flatten layer and needs far fewer parameters in the next layer, which reduces overfitting.
Image segmentation (G-2268)	Dividing an image into regions by saying which pixels belong to which object.
Pooling layer (G-1520)	A CNN layer that shrinks the feature map from a convolution layer by replacing each small window with one number, such as its maximum; this cuts the computation and makes the features less sensitive to small shifts.

Term	Meaning
Stride (G-1900)	The number of pixels the filter moves at each step; a stride larger than 1 makes the output smaller.
Filter (kernel) (G-777)	A small grid of weights slid over the image; at each position it multiplies and adds, so it responds strongly where its pattern (such as an edge) appears and produces a feature map. Its depth equals the input's number of channels.
Feature map (CNN) (G-766)	The grid of numbers a filter produces; large where the filter's pattern is present.
Receptive field (G-1642)	The area of the image that one cell (or one unit of a network) responds to.